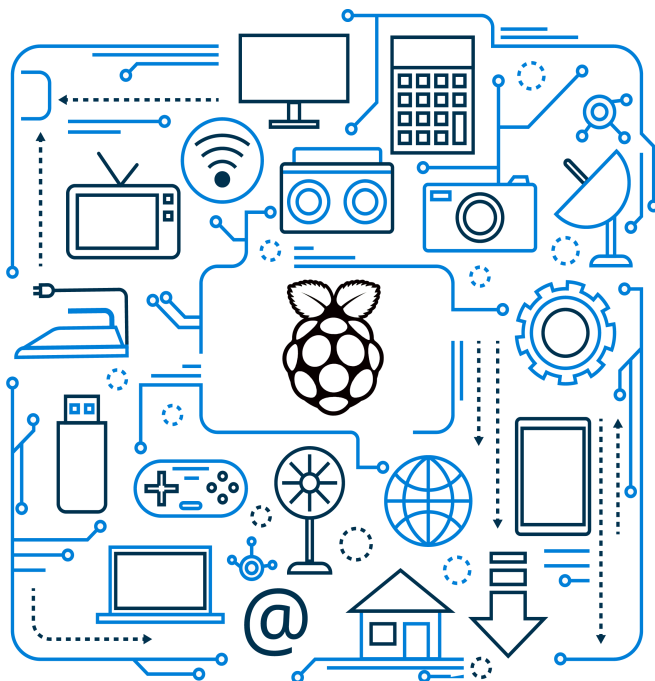


ALUMNE/A		DATA	
NIVELL	SDAMV2A	NOTA	
MATÈRIA	Programació de serveis i processos		
PROFESSOR/A	Gual Purí, Jordi		
Títol Activitat:	SDAMV2.2025.MP0490.T2-Projete-ServidorWebRaspberry		
		[] No aplicable [X] Aplicable	



Projecte SDAMV2 MP.0490

“SERVIDOR WEB RASPBERRY PI”

Aquest projecte té com a objectiu aplicar, en un cas pràctic, la programació de serveis de xarxa. En aquest cas concret, desenvoluparem un servidor, sobre la plataforma Raspberry Pi, per permetre monitorar la temperatura d'un espai a través del protocol HTTP.

A més d'això, aprofitarem aquest projecte per introduir alguns aspectes complementaris associats a tecnologies del món IoT.

SERVIDOR WEB RPI

FASE 1

F1.1

PREPAREM LA INSTAL·LACIÓ DEL SISTEMA OPERATIU PER AL SBC RASPBERRY PI

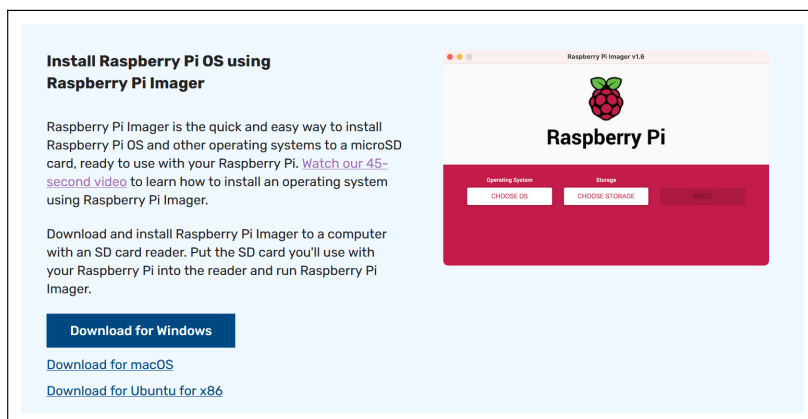
Per començar, necessitem tenir l'entorn de treball a punt. Raspberry Pi és una família de SBCs (Single-Board Computer) desenvolupat per la Raspberry Pi Foundation juntament amb el fabricant de semiconductors Broadcom, que poden ser utilitzats en projectes IoT, ja que disposen d'un conjunt bastant ampli d'interfícies de connexió per a dispositius electrònics. Els SBCs són un tipus de dispositius que requereixen que se'ls instal·li un sistema operatiu abans de poder ser utilitzats. Això és el que farem en aquest primer punt.

Per fer la instal·lació d'un sistema operatiu en un Raspberry Pi, cal preparar el procés en una targeta microSD, ja que aquest és el medi habitual d'instal·lació i d'emmagatzematge del dispositiu un cop instal·lat el sistema operatiu. Per preparar el medi d'instal·lació, seguiu els passos següents:

- Visiteu la pàgina web oficial de Raspberry Pi:

<https://www.raspberrypi.com>

- Descarregueu-vos l'aplicació Raspberry Pi Imager que trobareu a l'apartat Software d'aquesta pàgina web (la versió que correspongui al sistema operatiu de l'ordinador on fareu la preparació de la targeta microSD).



- Feu-ne la instal·lació (aquest procés no té cap aspecte a destacar).

- d. Un cop finalitzada la instal·lació, podeu executar l'aplicació.
- e. Connecteu la targeta microSD que se us ha donat, al vostre ordinador. Si en el moment de connectar-la us apareixen missatges dient que cal formatar-la, cancel·leu-los tots i tanqueu totes les finestres d'explorador d'arxius que se us obrin. El Raspberry Pi Imager ja crearà i formatarà totes les particions necessàries per fer la instal·lació.
- f. Escolliu Raspberry Pi 4 com a dispositiu (device), Raspberry Pi OS (64-bit) com a sistema operatiu i la targeta microSD que ens detecta l'aplicació.
- g. Premeu el botó SEGÜENT.
- h. Quan arribeu al pas de personalització, no premeu el botó d'ometre i aneu configurant els diversos aspectes que es demanen.
- i. Configureu les opcions següents amb els valors indicats (la resta, deixeu-les amb els seus valors predeterminats):
- **Nom de màquina:** rpi-grup-x (on "x" és el vostre número de grup).
 - **Configuració de Localització:**
 - Fus horari:** Europe/Madrid
 - Teclat:** ES
 - **Set username and password** (poseu tots el nom i password següents):
 - Username:** pi
 - Contrasenya:** raspberry
 - **Configuració wifi:**
 - SSID:** Joviat Alumnes
 - Contrasenya:** alumnes.joviat.wifi
 - País de la wifi:** ES
 - **Configuració wifi:**
 - SSID:** Joviat Alumnes
 - Contrasenya:** alumnes.joviat.wifi
 - **Configuració SSH:** de moment, deixeu-ho desactivat
 - **Configuració Raspberry PI Connect:** deixeu-ho desactivat
- j. Quan acabeu d'establir totes aquestes opcions, us apareixerà un resum final.

- i. Un cop fet tot això, podeu prémer el botó ESCRIU que farà que s'iniciï el procés de preparació de la targeta microSD (us demanarà confirmació abans de començar). Aquest procés trigarà uns minuts a finalitzar.
- j. Durant el procés i en el moment de finalitzar, és possible que us apareguin missatges dient que cal formatar la targeta microSD (cancel·leu-los) i que us apareguin finestres d'explorador d'arxius (tanqueu-les totes).
- k. Podeu extreure la targeta microSD de l'ordinador un cop acabat tot el procés.

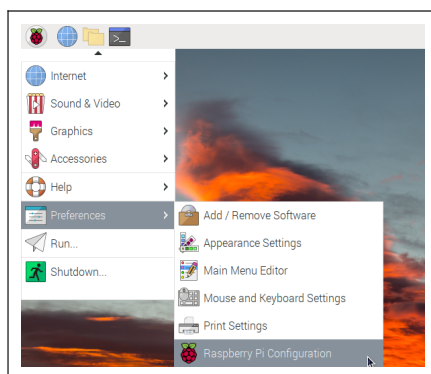
F1.2

Instal·lació del sistema operatiu al Raspberry Pi

Un cop tenim a punt el medi d'instal·lació, haurem de dur a terme el procés que ens portarà a tenir el nostre Raspberry Pi en condicions de ser utilitzat, amb un sistema operatiu Raspberry Pi OS, sobre el qual podrem instal·lar les diverses eines que necessitem per desenvolupar aquest projecte. El procés d'instal·lació del sistema operatiu consisteix a realitzar els passos següents:

- a. Sense endollar l'alimentació elèctrica, connecteu el teclat, el ratolí i el monitor al Raspberry Pi, i introduïu la targeta microSD que hem preparat a la ranura inferior que té el SBC.
- b. Connecteu la font d'alimentació al Raspberry Pi i poseu-lo en marxa. Durant la primera posada en marxa veureu que el monitor s'activa i es desactiva diverses vegades (és normal).
- c. Passats uns minuts després de posar en marxa el Raspberry Pi, pot ser que se'ns notifiqui que hi ha actualitzacions disponibles (sempre que la connexió a xarxa s'hagi pogut establir correctament). Si és el cas, donarem l'ordre d'instal·lar aquestes actualitzacions (el procés pot trigar uns minuts a finalitzar). En finalitzar direm que volem reiniciar.

- d. Abans de donar per acabat el procés de preparació de l'entorn de treball, ajustarem alguns aspectes addicionals de la configuració del sistema un cop s'hagi reiniciat. En primer lloc, obriu l'eina de configuració bàsica a la qual podem accedir amb l'opció de menú que es pot veure a continuació:



- e. Dins d'aquesta eina de configuració hi veurem 5 pestanyes, amb diverses categories d'opcions de configuració. Vegem-ne algunes:

- **System:** aquí podríem decidir iniciar el sistema amb entorn gràfic (To Desktop), o en entorn de comandes (To CLI). Ho deixem a To Desktop. També podríem desactivar l'Auto Login per fer que calgui validar-se com a usuari pi en el moment de la posada en marxa (com que estem en un entorn de proves, el podem deixar activat per comoditat).
- **Interfaces:** aquí ho desactivarem tot excepte la interfície 1-Wire que farem servir per al sensor de temperatura i la interfície Serial Console i Serial Port que han d'estar activades.
- **Localisation:** aquí podeu revisar les opcions de configuració L10N. Modificarem dos aspectes d'aquest apartat de configuració:

– **Locale:**

Language:	Català/Castellà/Anglès (escolliu)
Country:	ES (Spain)
Character Set:	UTF-8

– **Keyboard:**

Model:	Generic 105-key PC
Layout:	Spanish
Variant:	Catalan (Spain, with middle-dot L)

- f. Un cop confirmats els canvis, se'ns demanarà si volem reiniciar el sistema. De moment, diem que no.

- g. Obrim una finestra de comandes (icona  de la barra de tasques superior) i ens convertim en usuari **root** mitjançant l'execució de la comanda **sudo -i**. Un cop fet això, obrirem l'arxiu de configuració de la memòria swap del sistema amb les comandes:

```
root@rpi-grup-x:~# cd /etc/rpi
root@rpi-grup-x:/etc/rpi# mousepad swap.conf
```

- h. Se'ns obrirà un editor de text que contindrà l'arxiu de configuració de la memòria swap. Amb molt de compte de no equivocar-nos, descomentem l'opció **RamMultiplier=2** (eliminant el caràcter **#** que té al principi), fent que quedi com es pot veure a continuació:
- i. Guardem els canvis (**File** → **Save**) i tanquem l'editor.
- k. Ara podeu tancar la finestra de comandes executant dues vegades consecutives la comanda **exit** i, un cop fet això, podeu reiniciar el sistema (menú **Raspberry** → opció **Shutdown** → opció **Reboot**).

F1.3

INSTAL·LACIÓ DE LES EINES DE TREBALL

Un cop tenim el sistema operatiu bàsic instal·lat i amb la configuració bàsica resolta, passem a instal·lar les diverses eines que necessitarem per implementar el projecte que se us proposa. Per fer-ho, seguiu els passos que s'indiquen a continuació:

- a. Obriu una finestra de comandes i convertiu-vos en usuari **root** executant la comanda **sudo -i** com ja hem fet anteriorment.
- b. Actualitzeu els índexs dels repositoris de software del sistema executant la comanda:

```
root@rpi-grup-x:~# apt update
```

- c. Un cop fet això, passem a fer la instal·lació de l'entorn de desenvolupament **PyCharm**. Per fer-ho, obriu el navegador en el Raspberry Pi i accediu a l'URL:

<https://www.jetbrains.com>

- d. Obriu l'opció **Developer Tools** i seleccioneu **PyCharm**.
- e. A l'inici d'aquesta secció, veureu el botó **Download**. Premeu-lo.

- f. A la part superior d'aquest apartat de descàrregues, seleccioneu l'opció Linux.
- g. Baixeu a la secció on es pot descarregar l'edició Community Edition d'aquest entorn de desenvolupament. En el desplegable on apareix com a opció predeterminada .tar.gz, seleccioneu .tar.gz (Linux ARM64), ja que ARM64 és l'arquitectura hardware de Raspberry Pi. Un cop seleccionada aquesta opció s'iniciarà automàticament la descàrrega.
- h. Aneu a la finestra de comandes i situeu-vos al directori de descàrregues de l'usuari pi, que és el lloc on haurà quedat guardat l'arxiu que hem descarregat.

```
root@rpi-grup-x:~# cd /home/pi/Downloads
```

- i. Executeu la comanda següent per descomprimir l'arxiu que hem descarregat (el nom de l'arxiu podria canviar lleugerament si hem descarregat una versió diferent):

```
root@rpi-grup-x:/home/pi/Downloads# tar -zxvf pycharm-community-2023.3.3-aarch64.tar.gz
```

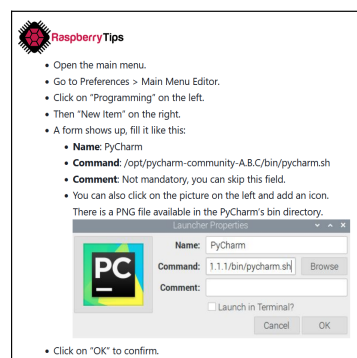
- j. Un cop finalitzada la descompressió (que pot trigar uns minuts) ens haurà aparegut un directori anomenat pycharm-community-2023.3.3 dins la carpeta Downloads.
- k. Ara haurem de moure aquest directori a la carpeta /opt, que és el lloc on típicament s'instal·len les aplicacions de tercers en sistemes Linux. Per fer-ho, executarem la comanda següent:

```
root@rpi-grup-x:/home/pi/Downloads# mv pycharm-community-2023.3.3 /opt
```

- l. Ara ja podem eliminar l'arxiu que hem descarregat per alliberar els aproximadament 0,5 gigabytes d'espai que ocupa:

```
root@rpi-grup-x:/home/pi/Downloads# rm pycharm-community-2023.3.3-aarch64.tar.gz
```

- m. Creeu un accés directe per executar PyCharm des del menú principal seguint els passos següents:



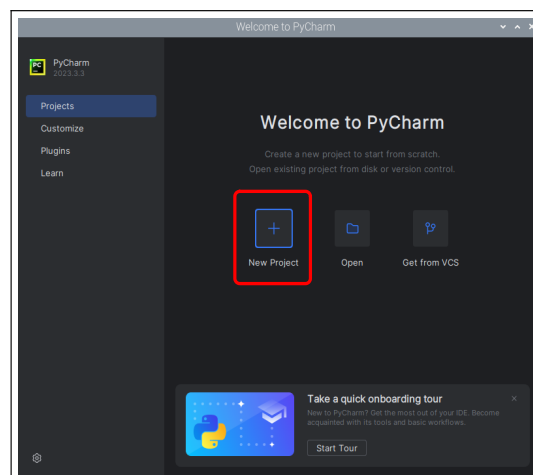
- n. Un cop fet això, si ho voleu, podeu crear també un accés directe a l'escriptori per poder obrir PyCharm d'una forma més ràpida.
- o. Com que el sistema operatiu de Raspberry Pi ja porta instal·lat per defecte l'interpret de Python i el gestor de paquets pip, per ara, no caldrà instal·lar res més.

F1.4

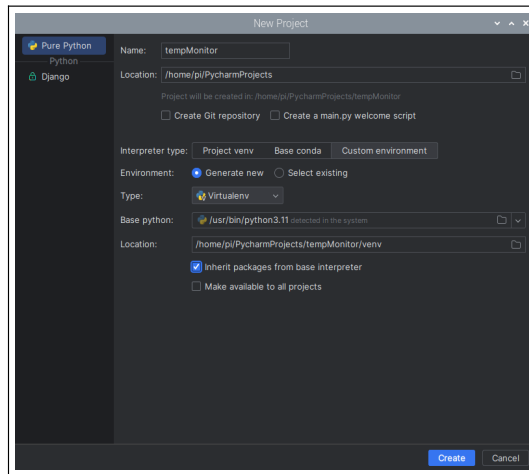
CREACIÓ D'UN PROJECTE PYCHARM

Un cop tenim les eines bàsiques de treball instal·lades, anem a veure com podem crear un projecte de desenvolupament per a una aplicació Python sobre l'entorn PyCharm:

- a. Obriu PyCharm i seleccioneu l'opció New Project:



- b. Ens apareixerà una finestra d'opcions per configurar el nou projecte. En aquesta finestra, establim les opcions següents:
 - Donarem nom al nou projecte.
 - A l'opció Interpreter type seleccionarem Custom environment.
 - Activarem l'opció Inherit packages from base interpreter.
 - Deixarem la resta d'opcions al valor que apareix de forma predeterminada.



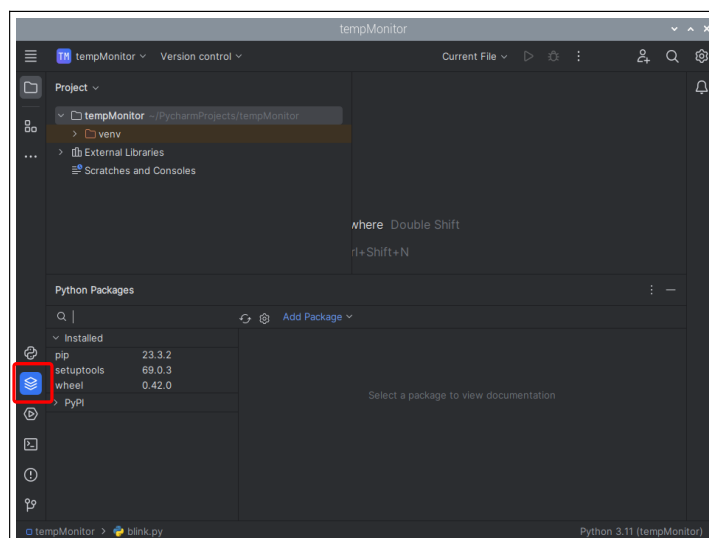
- c. Premem el botó Create i deixem el temps necessari perquè es faci tot el procés de creació del nou projecte (pot trigar una mica).

F1.5

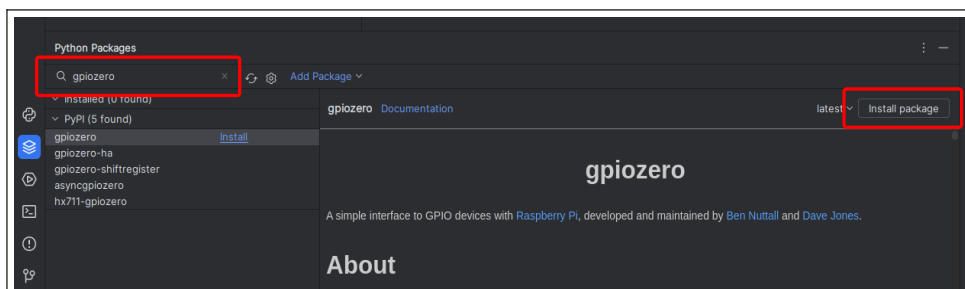
CONFIGURACIÓ DE LES DEPENDÈNCIES DEL PROJECTE

Un cop tenim el projecte creat anem a veure com el podem configurar perquè pugui fer ús de les llibreries bàsiques que ens han de permetre interactuar amb les diverses interfícies de control de dispositius de Raspberry Pi. Per fer-ho, seguim els passos següents:

- a. Seleccionem la icona  corresponent al gestor de packages que trobareu a la part inferior esquerra de l'entorn:



- b. A la casella de cerca indiqueu que voleu buscar el mòdul `gpiozero` i, quan us aparegui a la llista de resultats, cliqueu l'opció `Install package` (això equival a executar la comanda `pip install gpiozero`):



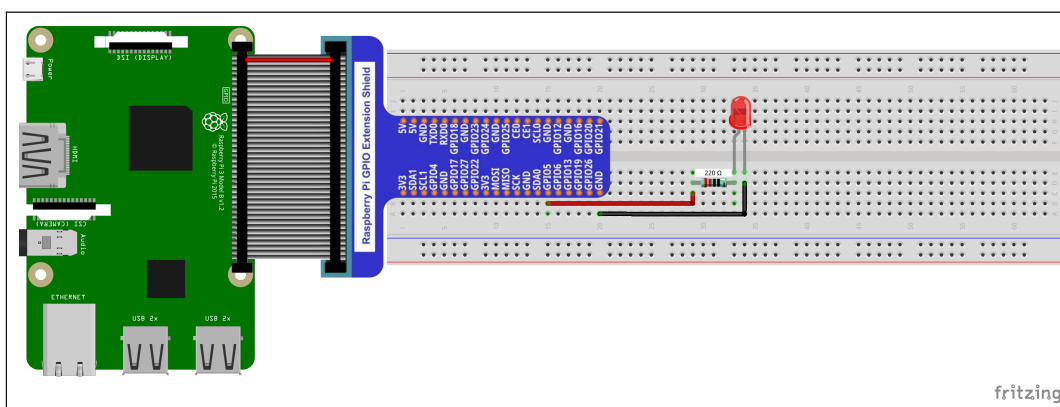
- c. El mòdul `gpiozero` ens ofereix un conjunt de classes que ens permeten controlar de forma molt senzilla els dispositius bàsics que podem connectar als GPIO del Raspberry.
- d. Repetiu el mateix procediment per instal·lar els mòduls `RPi.GPIO` i `lgpio`, que també ens ofereixen classes i funcions per al control dels GPIOs.
- e. Un cop fet això, ja tindrem a punt el nostre projecte per fer les primeres proves.

F1.6

MUNTATGE DEL CIRCUIT DE PROVA INICIAL

Abans de poder començar a escriure algun programa de proves, cal haver implementat un muntatge mínim amb dispositius connectats al Raspberry Pi. La primera prova que farem consisteix a posar en marxa el “hello world” típic en l'aprenentatge de programació en entorns IoT: l'anomenat “blink” que implementa el parpelleig d'un LED. Seguiu els passos següents:

- a. Amb el material que se us ha proporcionat, implementeu el muntatge que podeu veure a continuació, tenint el Raspberry Pi apagat (sense estar connectat al corrent elèctric):



- b. Un cop implementat aquest muntatge, poseu en marxa el Raspberry Pi.
- c. Per comprovar si s'ha fet correctament, un cop iniciat el sistema, obriu una finestra de comandes i executeu la comanda següent:

```
pi@rpi-grup-x:~ $ pinctl set 5 op dh
```

- d. Aquesta comanda, configura el GPIO número 5 del Raspberry Pi (el GPIO on tenim connectat el LED) com a GPIO de sortida (op → output), i el posa en mode "drive to high" (dh) que fa que en aquest GPIO hi hagi un valor alt de voltatge (+3,3V en Raspberry Pi), cosa que fa que s'encengui el LED. Si no s'encén en LED, reviseu bé el circuit i repasseu que hàgiu escrit bé la comanda.
- e. Si tot funciona correctament, podeu apagar el LED executant la comanda següent, on dl

```
pi@rpi-grup-x:~ $ pinctl set 5 op dl
```

significa "drive low" (posa 0V al GPIO número 5, apagant així el LED):

- f. Un cop feta la comprovació de funcionament del LED, obriu el PyCharm. Se us obrirà tal com el vam deixar el darrer cop que el vàreu utilitzar, amb el projecte que havíem creat anteriorment.
- g. Creeu un nou arxiu anomenat `blink.py` dins del projecte i poseu-hi el codi font que teniu a continuació:

```
1 import gpiozero as gz
2 import time
3
4 if __name__ == "__main__":
5
6     # Creem un objecte LED configurat al GPIO 5.
7     led = gz.LED(5)
8
9     # Encendrem i apagarem el LED 10 cops a intervals d'1 segon.
10    for i in range(10):
11        led.on()
12        time.sleep(1)
13        led.off()
14        time.sleep(1)
```

- h. Si tot ha anat bé, quan executeu aquest programa veureu com el LED vermell del circuit s'encén i s'apaga 10 vegades a intervals d'un segon. Molt probablement, us apareixeran alguns missatges de warning mentre s'executa el programa a la consola d'execució de PyCharm. Malgrat això, tot hauria de funcionar correctament.

- i. En aquest exemple, podem veure com la llibreria `gpiozero` ens proporciona una classe anomenada `LED`. Aquesta classe té un constructor al qual li diem el GPIO on tenim connectat el LED que volem controlar amb l'objecte que crearem i, un cop creat, podem controlar-lo d'una forma extremadament senzilla amb els mètodes `on()` i `off()`.

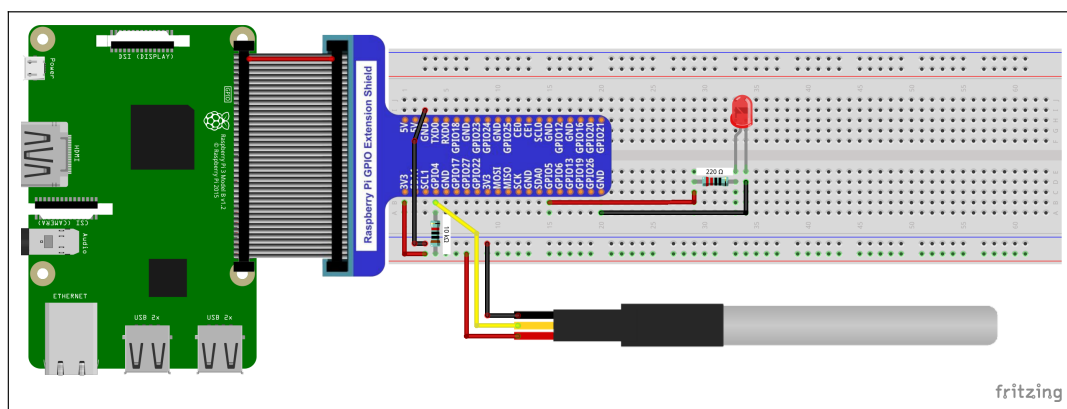
F1.7

MUNTATGE DEL CIRCUIT AMB EL SENSOR DE TEMPERATURA

Un cop hem arribat aquí, si tot el que hem proposat funciona correctament, voldrà dir que tenim l'entorn de treball correctament preparat i configurat per poder avançar.

El següent pas serà instal·lar el sensor de temperatura en el circuit que tenim i modificar el programa de control per fer que sigui capaç de prendre mostres d'aquest sensor i poder, així, determinar la temperatura a la qual es troba l'espai on el tenim ubicat. Els passos següents ens indiquen com dur a terme tot això:

- a. Amb el material que se us ha proporcionat a cada grup, implementeu el muntatge que podeu veure en el diagrama següent, tenint el Raspberry Pi apagat (sense estar connectat al corrent elèctric):



- b. Un cop implementat aquest muntatge, poseu en marxa el Raspberry Pi.
- c. Seguint el procediment d'instal·lació de packages en PyCharm explicat anteriorment, instal·leu el mòdul anomenat `w1ThermSensor` que facilita l'obtenció de mostres de diversos models de sensors de temperatura, entre els quals, el que tenim nosaltres (es tracta d'un sensor Dallas DS18B20).
- d. Creeu un nou arxiu anomenat `temperatura.py` dins del projecte i poseu-hi el codi font que teniu a continuació:

```
1 import sys
2 import gpiozero as gz
3 import withthermsensor as wt
4 import time
5 import datetime as dt
6
7 LED_VERMELL = 5
8
9 if __name__ == "__main__":
10
11     # Preparem els objectes necessaris per controlar el LED i el sensor.
12     sensor = wt.W1ThermSensor()
13     led = gz.LED(LED_VERMELL)
14
15     # Assegurem que el LED es trobi apagat i mostrem missatge inicial.
16     led.off()
17     print("Iniciant monitor de temperatura...")
18
19     # Bucle infinit que anirà prenent mostres de temperatura.
20     fi = False
21     while not fi:
22         try:
23             # Prenem la mostra, registrem data i hora i ho mostrem.
24             temperatura = sensor.get_temperature()
25             data_hora = dt.datetime.today().strftime("%Y/%m/%d %H:%M:%S")
26             print("%s - Temperatura: %.2f" % (data_hora, temperatura))
27
28             # Comprovem si la temperatura és major o igual que 25.
29             if temperatura >= 25:
30                 # Si ho és, encenem el LED.
31                 led.on()
32             else:
33                 # Si no ho és, apaguem el LED.
34                 led.off()
35
36             # Fem una pausa de 5 segons.
37             time.sleep(5)
38
39         except wt.errors.SensorNotReadyError as snre:
40             # Si es produeix un error de lectura del sensor, ho indiquem, però
41             # continuarem amb el bucle.
42             print("Error llegint el sensor. Continuem...", file=sys.stderr)
43
44     # Tancaments.
45     led.close()
46     print("Monitor de temperatura finalitzat.")
```

- e. Si executeu aquest programa tenint-ho tot ben muntat i configurat, veureu com entra en un bucle infinit en el qual, periòdicament, va llegint un nou valor de temperatura del sensor, el mostra per consola i, si el valor és més gran o igual que 25, encén el LED (en cas contrari, l'apaga).
- f. En el codi font de l'exemple que hem donat, hi podem veure alguns aspectes d'interès que comentem a continuació:
- **sensor = wt.W1ThermSensor()** → amb això es crea un nou objecte de tipus W1ThermSensor, que ens permetrà controlar el sensor de temperatura. És la forma més senzilla de fer-ho quan tenim un únic sensor connectat al pin número 4, que és el pin que ve configurat per defecte per connectar-hi dispositius que funcionen

amb el protocol 1-Wire (el sensor que tenim és un DS18B20, un dels models de sensor de temperatura que funciona amb el sistema 1-Wire).

Si tinguéssim més d'un sensor d'aquest tipus connectat al Raspberry Pi, hauríem d'utilitzar el class method `W1ThermSensor.get_available_sensors()` que retorna una llista amb tots els sensors que s'hagin pogut detectar.

- `temperatura = sensor.get_temperature()` → pren una mostra de temperatura a través del sensor i la retorna en graus centígrads. Durant la lectura de mostres es pot produir l'error `SensorNotReadyError` que cal tractar, ja que si no ho fem, en cas de produir-se, finalitzarà el programa de forma abrupta.



RQ.01 Un cop aconseguíu que funcioni correctament el muntatge amb el sensor de temperatura i el LED, juntament amb aquest darrer programa que va llegint mostres de la temperatura i que encén i apaga el LED, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

AVALUACIÓ DE LA FEINA FETA EN LA PRIMERA FASE DEL PROJECTE

La feina d'aquesta primera fase del projecte l'heu de fer en els grups de 3 alumnes que s'han designat. L'avaluació d'aquesta primera fase del projecte es farà sobre la revisió final de l'apartat **f1.7.g**. Per avaluar aquesta revisió, s'utilitzarà la rúbrica que teniu a continuació:

Aspecte	Ponderació sobre la nota de revisió
VALORACIÓ GLOBAL DE FUNCIONAMENT El circuit i el programa de control funcionen correctament en tots els aspectes que es demanen a l'enunciat, i ho fan de forma robusta: sense interrupcions imprevistes, podent utilitzar les diverses funcions repetidament, etc.	70%
VALORACIÓ HARDWARE Els components electrònics estan col·locats de forma lògica, ordenada i coherent (agrupant-los segons la seva funció, si s'escau) sobre la placa de proves, facilitant així la interpretació i comprensió del circuit. La connexió entre components mitjançant els cables jumper està feta de forma ordenada, seguint una sistemàtica de colors coherent i sense provocar més encreuaments dels que siguin estrictament necessaris.	30%

SERVIDOR WEB RPI

FASE 2

F2.1

PUBLIQUEM LES LECTURES DE TEMPERATURA VIA HTTP/HTML

Amb el muntatge hardware bàsic en funcionament, passem a la fase següent que consistirà a convertir el nostre projecte en un projecte IoT. Entenem que els projectes IoT han de proporcionar algun tipus de servei (de consulta, de control o d'ambdues coses) a través d'internet. Això és el que afegirem en aquesta segona fase.

Per fer-ho, utilitzarem el mòdul `http.server` que forma part del nucli de Python (no cal fer cap instal·lació específica), que permet convertir els nostres programes en servidors de protocol HTTP o HTTPS. Seguiu els passos que s'indiquen a continuació, que ens permetran posar en marxa un servidor senzill HTTP de prova i, alhora, ens permetran aprendre el funcionament d'aquesta biblioteca:

- a. El circuit electrònic no cal modificar-lo (de fet, en aquest primer exemple no el farem servir). Per tant, deixeu-lo tal com el teniu de la fase anterior.
- b. El procediment bàsic per preparar un servidor web en Python consta dels passos que es descriuen a continuació:
 - Crear una classe pròpia, subclasse de `BaseHTTPRequestHandler`, que actuarà com a gestor del nostre servidor. En aquesta classe sobreescrivem els mètodes corresponents a les operacions HTTP per les quals vulguem que el nostre servidor tingui respostes programades: `do_GET()`, `do_POST()`, etc.
 - Crear un objecte `HTTPServer`, indicant el nom o IP i el port en els quals el nostre servidor atindrà les peticions HTTP i associant-li la classe pròpia que hem creat per fer de gestor de peticions.
 - Executar el mètode `serve_forever()` de l'objecte `HTTPServer` que hem creat per entrar en un bucle infinit de servidor en el qual s'aniran atenent de forma indefinida les peticions HTTP que puguem rebre.
 - Si volem que es faci una finalització ordenada del servidor quan se l'interrompi de forma forçada, caldrà fer el `try/except` corresponent.

c. Observeu el primer exemple d'aplicació Python que implementa un servidor web:

```
1 import http.server as ws
2
3 # Definim els paràmetres de l'endpoint on tindrem el servidor a l'escolta.
4 HOST_NAME = "127.0.0.1"
5 PORT = 8080
6
7 # Classe pròpia que implementa el gestor de peticions HTTP del nostre
8 # servidor. Gestionarem únicament les peticions amb el mètode GET.
9 class ServerPropi(ws.BaseHTTPRequestHandler):
10
11     def do_GET(self):
12         if self.path == "/favicon.ico":
13             # Si ens demanen el favicon, retornarem error "not found".
14             self.send_response(404, "Not found")
15             self.end_headers()
16         else:
17             # Per la resta de peticions, retornarem un contingut HTML bàsic.
18             self.send_response(200, "OK")
19             self.send_header("Content-type", "text/html")
20             self.end_headers()
21             resposta = """
22                 <!DOCTYPE html>
23                 <html>
24                     <head>
25                         <meta charset="utf-8">
26                         <title>Servidor bàsic HTTP propi</title>
27                     </head>
28                     <body>
29                         <h1>Això és un servidor web de proves</h1>
30                     </body>
31                 </html>
32             """
33             self.wfile.write(bytes(resposta, "utf-8"))
34             self.wfile.flush()
35
36 if __name__ == "__main__":
37     # Preparem l'objecte HTTPServer que implementarà el servidor.
38     web_server = ws.HTTPServer((HOST_NAME, PORT), ServerPropi)
39     print("Servidor engegat -> http://%s:%s" % (HOST_NAME, PORT))
40
41     # Posem en marxa el servidor, gestionant la seva finalització ordenada.
42     try:
43         web_server.serve_forever()
44     except KeyboardInterrupt:
45         pass
46
47     web_server.server_close()
48     print("Servidor aturat.")
```

d. Aquest servidor, per cada petició GET que rep sobre el port 8080, comprova si l'URI de la petició és `/favicon.ico`, o no:

- Si ho és, retorna una resposta amb l'status code 404.
- Si no ho és, retorna la resposta HTML que tenim preparada.

e. Si voleu saber per què cal controlar aquest cas concret de les peticions que es fan sobre l'URI `/favicon.ico`, podeu consultar l'adreça següent:

<https://en.wikipedia.org/wiki/Favicon>

- f. Un cop vist el procediment de crear un servidor HTTP bàsic amb Python, heu de crear-ne un de nou que permeti consultar la temperatura del sensor via web. Aquest nou servidor que heu de crear, per cada petició de connexió GET que rebi sobre el port 8080, haurà de:
- Fer la comprovació, com hem vist anteriorment, sobre si la petició que s'ha rebut és sobre el recurs `/favicon.ico`. En cas afirmatiu, retornar l'error 404. En cas negatiu, continuar amb el procés normal (passos següents).
 - Consultar, a través del sensor, el valor actual de temperatura.
 - Preparar un petit fragment de codi HTML que maqueti una senzilla pàgina web que permeti mostrar el valor de la temperatura que haurem llegit del sensor.
 - Retornar una resposta HTTP al client que contindrà, en el seu cos, el codi HTML que haurem preparat, que s'acabarà visualitzant en el navegador del client.
- g. Investigueu què cal posar com a adreça del servidor per aconseguir que s'atenguin les peticions que puguin venir des del navegador de la mateixa Raspberry i, també, les peticions que puguin venir de clients remots.



RQ.02 Un cop tingueu feta aquesta primera versió del servidor, que implementi aquest procediment senzill que hem descrit per atendre els clients que es connectin, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

F2.2

FEM UNA ANÀLISI CRÍTICA DE LA PRIMERA VERSIÓ DEL SERVIDOR

Havent completat l'apartat F2.1, hem obert la porta a poder consultar remotament les mesures de temperatura del nostre sensor i, tenint en compte que ho fem a través del protocol HTTP, amb la configuració adient, es podria arribar a fer des de qualsevol ubicació geogràfica des d'on puguem disposar de connexió a internet.

Però, si fem una senzilla prova de càrrega o estrès d'aquesta primera versió del servidor, generant moltes connexions de consulta en un interval de temps molt petit, veurem que apareixen problemes significatius per poder atendre correctament i d'una forma ràpida totes les peticions. Seguint els passos següents, intentarem fer evident aquest problema, diagnosticar quina n'és la causa i pensar possibles solucions:

- a. Per evidenciar que la versió actual del servidor té aquest problema de poca resistència a una càrrega gran de peticions, des dels ordinadors de dos dels components del grup genereu repetides peticions de consulta de la temperatura utilitzant un navegador. Feu la primera consulta i, un cop feta, deixeu premuda la tecla F5 que en la majoria dels navegadors és la que provoca una actualització de la pàgina actual, generant una nova petició sobre el mateix URL que teníem (si la deixeu premuda, es generaran un munt de connexions en un interval reduït de temps).
- b. Observeu com, de bon principi, ja notem que moltes de les peticions experimenten un notable retard en la seva resposta i, fins i tot, és probable que es generin errors d'execució en el servidor que acabin provocant, també, errors en el navegador del client.
- c. Analitzeu quina és la causa del problema. Podeu posar, per exemple, missatges de log per consola en diferents punts del codi del servidor per veure en quin/s punt/s s'encalla l'execució del codi que genera la resposta a les peticions dels clients.
- d. Una vegada tots els grups hagin fet la seva diagnosi, les posarem en comú i discutirem quina és la raó concreta que provoca el problema.
- e. A partir d'aquí, s'obrirà un espai de temps de 20–30 minuts perquè cada grup pugui fer una proposta de solució (proposta d'idees sense implementar-les).
- f. Passat aquest temps s'obrirà un procés de debat entre els diferents grups per exposar i defensar les idees de solució proposades.

- g. Un cop discutides, s'haurà de passar a la seva implementació, aplicant alguna de les que s'hagi considerat com a viable.



E.01 / RF.01 Quan cregueu que heu fet una diagnosi correcta de la causa del problema, feu-ne una explicació per escrit en un document PDF que haureu de penjar a Moodle. Un cop fet això, aviseu el professor perquè us revisi l'explicació que heu fet per valorar la seva correctesa i precisió.

F2.3

IMPLEMENTACIÓ DE MILLORES A LA VERSIÓ INICIAL: SERVIDOR.V2

Havent fet una anàlisi dels problemes que presenta la versió inicial del servidor passarem a resoldre'ls implementant una de les possibles solucions que hem proposat. Abans de fer-ho, però, posem negre sobre blanc el diagnòstic i la solució que hem identificat a la fase anterior:

- **Diagnosi (identificació de l'arrel del problema):** el problema de la versió inicial del servidor rau a la línia de codi on es fa la lectura de la temperatura que està mesurant el sensor:

```
sensor.get_temperature()
```

La crida al mètode `get_temperature()` provoca un retard de, com a mínim, 750 mil·lisegons, que és el temps que necessita el circuit controlador del sensor per fer la conversió analògica-digital (es pot reduir aquest temps disminuint l'exactitud de les mesures, però, en qualsevol cas, es converteix en un retard prohibitiu).

- **Efecte del retard:** el retard, tot i no semblar molt elevat (és de l'ordre de mil·lisegons), es converteix en un problema quan el que pretenem és implementar un servidor web. Si, per exemple, suposem que en un moment donat ens arriben 100 connexions en un interval de 30 segons, comptant que cada connexió requereix llegir la temperatura del sensor, veurem que caldran 75 segons per atendre-les totes pel cap baix ($750 \text{ ms} \cdot 100 = 75 \text{ s}$).

Per tant, en un escenari com aquest, hi hauria un cert nombre de connexions que experimentarien un retard de l'ordre de desenes de segons a ser ateses independentment de la latència i l'amplada de banda de la connexió de xarxa (això, avui dia, és inconcebible).

- **Proposta de solució:** la proposta de solució que adoptarem per eliminar aquest problema consisteix a:

- Continuarem tenint el bucle infinit que va atenent connexions, el que iniciem amb el mètode `serve_forever()`, en el thread principal del servidor.
- Farem que el servidor web sigui multiclient concurrent. És a dir, que per cada petició de connexió que rebem es creï un nou thread que atengui el client que s'acaba de connectar. Això s'aconsegueix, simplement, canviant la classe que implementa el servidor web de `HTTPServer` a `ThreadingHTTPServer`:

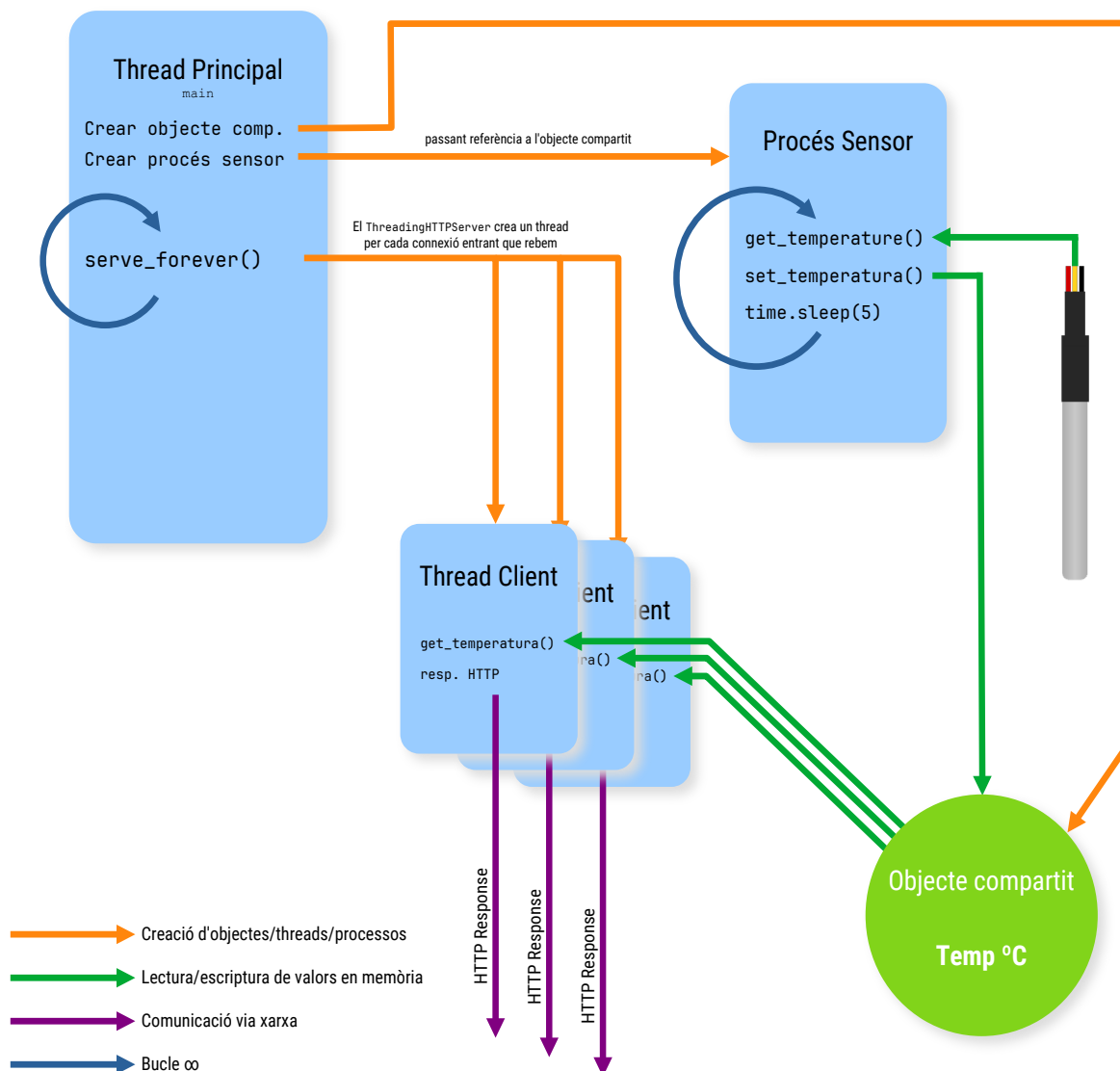
```
1 import http.server as ws
2
3 # Definim els paràmetres de l'endpoint on tindrem el servidor a l'escolta.
4 HOST_NAME = "127.0.0.1"
5 PORT = 8080
...
...
...
if __name__ == "__main__":
    # Preparam l'objecte ThreadingHTTPServer que implementarà el servidor.
    web_server = ws.ThreadingHTTPServer((HOST_NAME, PORT), ServerPropi)
    print("Servidor engegat -> http://%s:%s." % (HOST_NAME, PORT))

    # Posem en marxa el servidor, gestionant la seva finalització ordenada.
    try:
        web_server.serve_forever()
    except KeyboardInterrupt:
        pass

    web_server.server_close()
    print("Servidor aturat.")
```

Amb això, també aconseguirem millorar el temps de resposta del servidor, ja que estarem aprofitant les possibilitats multithreading de la Raspberry Pi.

- Abans d'iniciar el bucle infinit del servidor web, crearem un procés auxiliar que s'executarà de forma contínua, el qual, cada cert temps (cada 5 segons, per exemple), llegirà la temperatura del sensor i la guardarà en un objecte compartit al qual s'hi haurà de poder accedir des de qualsevol punt de l'aplicació.
- Els threads que es vagin creant per atendre connexions HTTP, llegiran la temperatura de l'objecte compartit (sense experimentar cap mena de retard) en comptes de fer-ho directament del sensor.
- Cal dir que, malgrat totes les millores implementades, en cas d'una alta càrrega de connexions, continuarem experimentant alguns errors en el servidor, ja que la llibreria `http.server` no està pensada per donar les prestacions d'un servidor web d'alta capacitat. Però, en qualsevol cas, obtindrem una implementació molt més robusta i amb major capacitat d'atendre connexions en comparació amb el servidor bàsic que hem implementat inicialment.



Un cop entès quina és la causa del problema, els seus efectes i la proposta de solució, seguim els passos següents per implementar la versió 2 del servidor:

- Reestructureu el codi de la versió inicial del servidor per fer que es correspongui amb la proposta que hem descrit.
- En el moment que tingueu feta la implementació d'aquesta segona versió del servidor, feu proves de càrrega com vam fer amb la versió 1, generant moltes connexions simultànies. Comproveu que el rendiment del servidor ha millorat substancialment respecte de la versió anterior, malgrat que poden continuar apareixent error per sobrecàrrega.



RQ.03 Un cop tingueu implementada la versió 2 del servidor i hàgiu fet les proves de càrrega per assegurar que s'ha resolt el problema que teníem en la primera versió, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

F2.4

SERVIDOR.V3: MILLOREM EL SERVIDOR AMB COMUNICACIONS SEGURES HTTPS

Un cop tenim un servidor més robust que el de la versió inicial, ara toca posar-lo al dia en l'àmbit de la seguretat. Avui dia, és pràcticament inconcebible que un servidor HTTP no tingui la capacitat de treballar amb comunicacions segures, malgrat que la comunicació entre clients i servidor no ho requereixi. Per això, la següent millora consistirà a habilitar el protocol HTTPS (HTTP segur) en el nostre servidor i permetre així que les comunicacions entre clients i servidor siguin encriptades i, per tant, es proporcioni el servei de privacitat.

Per implementar l'ús de comunicacions segures mitjançant HTTPS generarem un certificat digital **autosignat** per al nostre servidor. Per aquest motiu, no estarem oferint la garantia d'autenticitat del servidor que sí que donaríem si utilitzéssim un certificat digital signat per una autoritat de certificació (cosa que té un cost econòmic). Però, com a activitat d'aprenentatge, ja ens servirà.

Per afegir la capacitat d'establir connexions segures via HTTPS al vostre servidor web, seguiu els passos que s'indiquen a continuació:

- Obriu una finestra de comandes.
- Amb la comanda **cd**, situeu-vos dins la carpeta principal del projecte que esteu desenvolupant, on teniu els arxius de codi font.
- Un cop fet això, amb la comanda **mkdir**, creeu una subcarpeta anomenada **cert** on guardarem els arxius de claus criptogràfiques i certificats digitals del servidor que crearem a continuació:

```
pi@rpi-grup-x:~/proj# mkdir cert
```

- d. El certificat digital per al nostre servidor el crearem mitjançant l'eina **openssl**, que la trobarem instal·lada de sèrie en el sistema operatiu de la Raspberry Pi. La creació la durem a terme executant la comanda següent que crearà un certificat digital autosignat que permetrà proporcionar privacitat en les comunicacions del servidor web, però **no autenticitat del servidor**. El procés de creació del certificat requereix respondre tot un seguit de preguntes, les respostes de les quals no cal que siguin reals, ja que estem creant un certificat de prova.

```
pi@rpi-grup-x:~/proj# openssl req -newkey rsa:4096 -x509 \
                        -keyout cert/key.pem -out cert/cert.pem \
                        -days 365 -nodes
```

- e. Un cop creats els arxius necessaris per poder convertir el nostre servidor d'HTTP a HTTPS, passarem a fer les modificacions corresponents al codi de l'aplicació.
- f. En primer lloc, caldrà crear un objecte `SSLContext` i configurar-lo perquè utilitzi els arxius que acabem de crear com a certificat de servidor. La classe `SSLContext` la trobareu al mòdul `ssl` de Python:

```
...

# Preparem l'objecte SSLContext que ens ha de permetre convertir el socket
# bàsic del servidor a socket SSL (TLS).
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="cert/cert.pem", keyfile="cert/key.pem")
context.check_hostname = False

...
```

- g. I, un cop fet això, caldrà convertir el socket bàsic del servidor a socket SSL que és el tipus de socket que permet implementar el protocol HTTPS:

```
...

# Fem la conversió del socket bàsic del servidor a socket SSL (TLS).
webServer.socket = context.wrap_socket(webServer.socket, server_side=True)

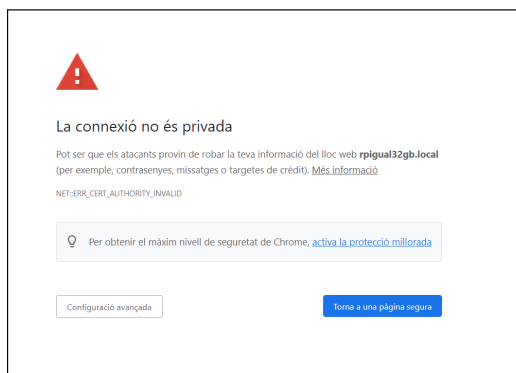
...
```

- h. Un cop fet això, podem continuar amb el procés normal engegant el bucle infinit del servidor amb el mètode `serve_forever()`.

- k. Només falta comentar que, quan ens vulguem connectar des d'un navegador, ara haurem d'especificar, explícitament, que ho volem fer mitjançant protocol HTTPS i a través del port que hàgim configurat (hem canviat el port a 8443, per analogia amb els ports estàndards de l'HTTP, que utilitza el 80, i HTTPS, que utilitza el 443):

https://ip_servidor_raspberry:8443

- l. Tingueu en compte que, com que estem treballant amb un certificat digital de servidor **autosignat**, cada cop que ens connectem per primera vegada des d'un determinat navegador, ens apareixerà un avís com el següent:



- m. Aneu a la configuració avançada i indiqueu que voleu continuar endavant de totes maneres. Això ho fem perquè coneixem l'autenticitat del servidor sobre el qual ens estem connectant (si ens trobéssim això quan ens connectem a un servidor d'internet del qual no estem segurs de la seva autenticitat, no hauríem de continuar endavant per raons de seguretat).

Continuant endavant malgrat l'avís, tindrem una connexió que ens proporciona privacitat (les comunicacions són encriptades), però no autenticitat del servidor.



RQ.04 Un cop tingueu implementada la versió 3 del servidor i hàgiu fet les proves de connexió per assegurar que la connexió ara es fa a través del protocol HTTPS en comptes de fer-ho per HTTP, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

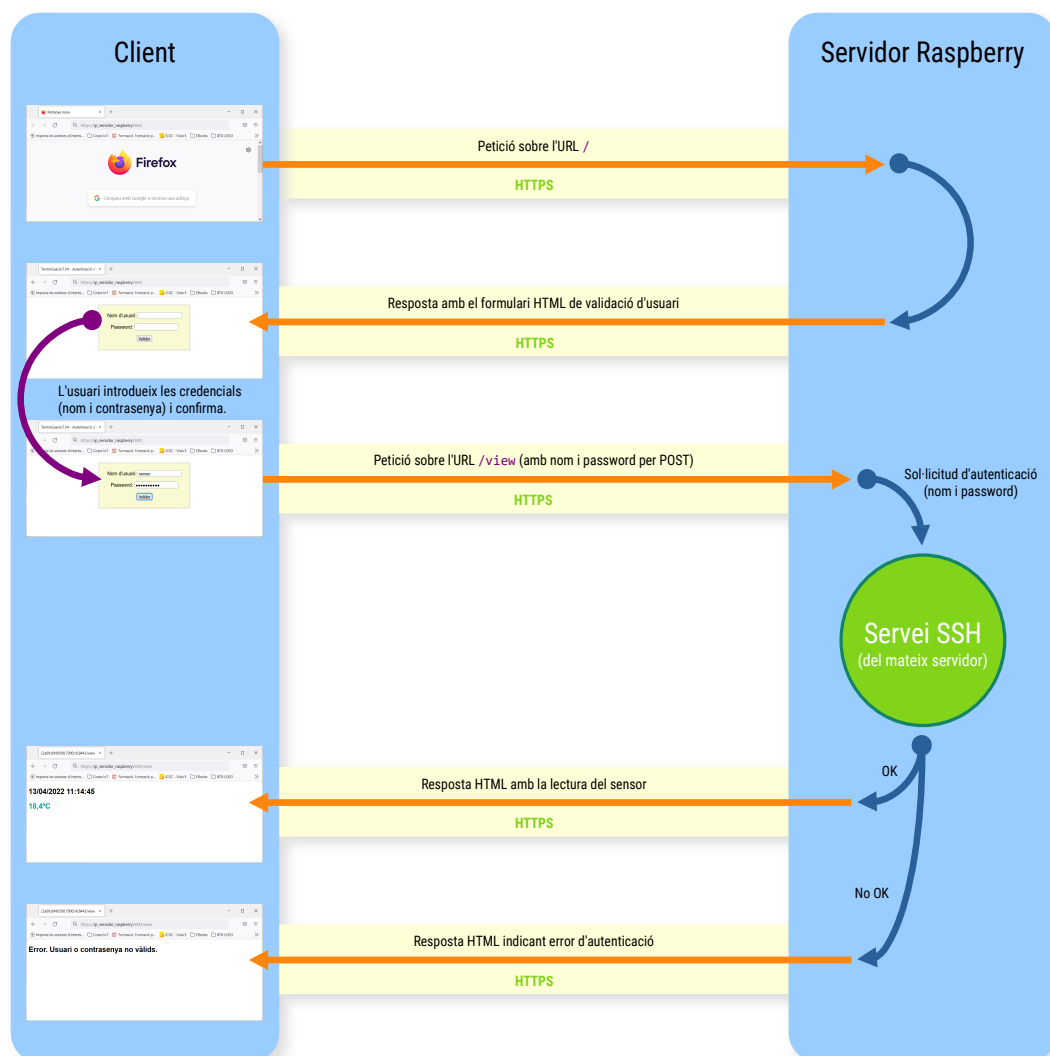
F2.5

SERVIDOR.V4: ASSEGUREM L'ACCÉS A LES DADES AMB AUTENTICACIÓ D'USUARIS

Habilitant les comunicacions via HTTPS aconseguim privacitat en la transmissió de les dades entre clients i servidor, però no proporcionem una protecció d'aquestes dades davant d'accessos no autoritzats realitzats directament sobre el servidor. Per fer-ho, haurem d'implementar algun sistema d'autenticació d'usuaris.

De sistemes d'autenticació d'usuaris, n'hi ha molts i diversos: des d'implementar un sistema propi gestionat pel mateix programa, passant per la validació a través d'una base de dades d'usuaris i passwords, fins a la utilització de protocols com SSH o LDAP. Donat que en el sistema operatiu Raspbian és molt senzill activar el servei SSH, utilitzarem aquest protocol per implementar el nostre sistema d'autenticació d'usuaris, cosa que ens permetrà utilitzar els comptes d'usuari del mateix sistema operatiu per autoritzar l'accés a les dades del sensor.

Per afegir aquesta funcionalitat al nostre servidor, per començar, caldrà desenvolupar una pàgina HTML que implementi un formulari de validació amb nom d'usuari i password. Aquesta pàgina de validació inicial serà la que ens portarà a executar la petició de visualització de les dades del sensor amb l'URL `/view`, passant els paràmetres, nom i contrasenya, per POST. I, abans de retornar la resposta HTML amb les mesures llegides del sensor, haurem d'utilitzar el servei d'autenticació d'SSH per determinar si l'usuari que demana consultar les dades és vàlid, o no. En definitiva, estarem implementant l'esquema d'autenticació següent:



Un altre aspecte que haurem de resoldre és l'obtenció, en el servidor, de paràmetres tramesos per POST d'una petició enviada per part d'un client. Quan s'envien credencials d'usuari a través d'una petició HTTP, a més de fer-ho utilitzant el protocol segur HTTPS és necessari enviar-los per POST. Si s'envien per GET, les credencials seran visibles en clar a la barra d'adreces del navegador del client, la qual cosa es considera un problema de seguretat greu.

Per obtenir els paràmetres que un client ens envia dins del contingut d'una petició feta amb el mètode POST haurem de seguir els passos que s'indiquen a continuació:

- a. En primer lloc, haurem de sobreescriure el mètode `do_POST()` en la classe que implementa el gestor de peticions del servidor HTTP:

```
1 import http.server as ws
2
3 # Definim els paràmetres de l'endpoint on tindrem el servidor a l'escolta.
4 HOST_NAME = "0.0.0.0"
5 PORT = 8443
6
7 # Classe pròpia que implementa el gestor de peticions HTTP del nostre
8 # servidor. Gestionarem únicament les peticions amb el mètode GET.
9 class ServerPropi(ws.BaseHTTPRequestHandler):
10
11     def do_GET(self):
12         # Resposta a les peticions rebudes via mètode GET.
13         if self.path == "/favicon.ico":
14             ...
15
16     def do_POST(self):
17         # Resposta a les peticions rebudes via mètode POST.
18         ...
```

- b. Quan rebem una petició HTTP per POST d'un client, i aquesta petició conté els valors introduïts en un formulari en forma de paràmetres, els podem obtenir mitjançant el patró de codi següent:

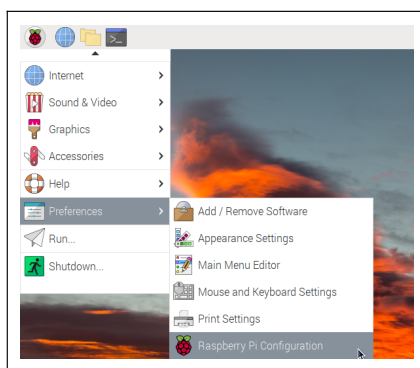
```
from urllib import parse
...
def do_POST(self):
    mida = int(self.headers.get("content-length"))
    dades = self.rfile.read(mida)
    parametres = parse.parse_qs(str(dades, "UTF-8"))
    ...
```

- c. En aquest exemple, la variable `parametres` és un diccionari que conté una parella clau-valor per cadascun dels valors que hem rebut del formulari. Si suposem que el formulari tenia un camp de tipus text anomenat `user`, per al nom d'usuari, i un camp de tipus password anomenat `passw`, per a la contrasenya, podrem obtenir ambdós valors de la forma que es pot veure a continuació:

```
...
parametres = parse.parse_qs(str(dades, "UTF-8"))
usuari = parametres["user"][0]
password = parametres["passw"][0]
...
```

- d. Si tot ha anat bé, les variables `usuari` i `password` contindran el nom d'usuari i la contrasenya, respectivament, que el client hagi introduït en el formulari de validació. Amb això, podrem passar a verificar-ne l'autenticitat.

Un cop tenim el nom d'usuari i la contrasenya en sengles variables de tipus String, podem passar a fer-ne l'autenticació sobre el mateix servidor Raspberry Pi mitjançant el protocol SSH. Per fer-ho, caldrà seguir els passos que s'expliquen a continuació:



- e. En primer lloc, obriu l'eina de configuració bàsica del sistema a la qual podem accedir amb l'opció de menú que es pot veure a continuació:
- f. Dins d'aquesta eina de configuració hi veurem 5 pestanyes. Haurem de seleccionar la pestanya Interfaces i, de la llista de serveis disponibles, haurem d'activar l'SSH (el primer de la llista), si és que no l'havíem activat prèviament.
- g. Confirmem els canvis prement el botó OK i reiniciem el sistema. Si no hem canviat la contrasenya predeterminada de l'usuari pi, ens avisarà que tenim un problema de seguretat, ja que activant el servei SSH estem habilitant les connexions externes sobre el nostre Raspberry Pi. Les connexions externes habilitades combinades amb passwords predeterminats fan un còctel explosiu en termes de seguretat informàtica. De totes maneres, trobant-nos en un entorn d'aprenentatge restringit a la xarxa local de l'escola, podeu deixar-ho així, si voleu.
- h. Quan hàgim reiniciat el sistema, passarem a crear un compte d'usuari per poder-lo utilitzar en el procés d'autenticació que requerirem per poder realitzar la consulta de les dades del sensor. Per fer-ho, obriu una finestra de comandes, convertiu-vos en usuari root amb la comanda `sudo -i`. Un cop tingueu privilegis d'administrador, amb la comanda `adduser`, doneu d'alta un nou usuari amb el nom que vulgueu (en l'exemple, el nom triat és sensor):

```
root@rpi-grup-x:~# adduser sensor
S'està afegint l'usuari «sensor»...
S'està afegint el grup nou sensor (1001)...
S'està afegint el nou usuari sensor (1001) amb grup sensor...
S'està creant el directori personal «/home/sensor»...
S'estan copiant els fitxers des de «/etc/skel»...
Nova contrasenya:
Torneu a escriure la nova contrasenya:
passwd: s'ha actualitzat la contrasenya satisfactòriament
S'està canviant la informació d'usuari per a sensor
Introduïu el nou valor, o premeu INTRO per al predeterminat
Nom complet []: Usuari sensor de temperatura
Número d'espai []:
Telèfon de la feina []:
Telèfon de casa []:
Altres []:
És aquesta informació correcta? [S/n] S
root@rpi-grup-x:~#
```

- i. Un cop fet tot això, ja tenim un usuari a punt i el servei SSH activat per poder realitzar proves d'autenticació des del programa del nostre servidor. Ja podeu tancar la finestra de comandes.
- j. Tornem al nostre projecte de PyCharm i, a continuació, mitjançant el gestor de packages, afegim el mòdul anomenat paramiko. Aquesta biblioteca de Python implementa el protocol SSHv2 per poder donar funcionalitats de client o de servidor a les aplicacions que l'utilitzen.
- k. Amb el següent code snippet es pot dur a terme la verificació de les credencials d'un usuari que tinguem guardades a les variables usuari i passwd, via protocol SSH, sobre el mateix servidor on s'executi el programa:

```
1 import paramiko
2
3 result = None
4 usuari = "..."
5 password = "..."
6
7 try:
8     with paramiko.SSHClient() as ssh_client:
9         # Si el certificat del servidor és autosignat, haurem de fer una
10         # verificació no estricta. Aquesta línia ho configura així.
11         ssh_client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
12
13         # Configurem les opcions de validació de l'usuari. Si fem la
14         # verificació sobre el mateix servidor on s'executa el programa, ens
15         # hem de connectar a "localhost"; si no, posarem la IP o nom de domini
16         # que correspongui.
```

```
17     ssh_client.connect("localhost", username=usuari, password=passwd)
18     result = usuari
19
20     except paramiko.AuthenticationException as ae:
21         # Si es produeix una AuthenticationException voldrà dir que no s'ha validat
22         # correctament l'usuari. Les altres excepcions es poden produir per altres
23         # motius (error de connexió, certificat incorrecte, etc.).
24         print("Error d'autenticació")
25
26     except Exception as e:
27         print("Error: %s" % e)
28
29     # Si l'autenticació s'ha fet correctament, la variable result haurà deixat de
30     # valer None i contindrà el nom d'usuari.
31     ...
```

Un cop vistes totes les qüestions que necessitem conèixer per implementar l'esquema de validació que s'ha proposat, amplieu la versió 3 del servidor (serà la versió 4) per fer que s'apliqui aquest sistema d'autenticació en el moment que un client vulgui consultar les dades del sensor. Per fer-ho, seguiu els passos que s'indiquen a continuació:

- l. Modifiquen la resposta del servidor per fer que, en rebre una petició sobre l'URL `/`, retorni el formulari de validació en comptes de les dades del sensor, de la manera que s'indica en el diagrama de l'esquema de validació que hem donat anteriorment.
- m. Feu que les dades del sensor es retornin quan es faci una petició sobre l'URL `/view` i, en aquest cas, feu que el servidor verifiqui les credencials d'usuari sobre ell mateix via protocol SSH utilitzant el nom d'usuari i password que hauríem de rebre com a paràmetres d'aquesta petició via POST.



E.02 / RQ.05 Un cop aconseguíu que funcioni correctament la versió 4 del servidor, amb autenticació d'usuari, per poder accedir a les dades del sensor, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara i feu l'entrega del projecte segons les indicacions que teniu a continuació.

AVALUACIÓ DE LA FEINA FETA EN LA SEGONA FASE DEL PROJECTE

La feina d'aquesta segona fase del projecte l'heu de continuar fent en els grups de 3 alumnes que s'han designat. A més dels punts de revisió i avaluació intermedis d'aquesta segona fase, en finalitzar l'apartat F2.5 caldrà entregar el projecte Python amb la versió 4 del servidor a través de Moodle. L'avaluació d'aquesta darrera entrega de la fase 2 es farà segons la rúbrica que teniu a continuació:

Aspecte	Ponderació
FUNCIONAMENT El circuit i el programa de control funcionen correctament en tots els aspectes que es demanen a l'enunciat, i ho fan de forma robusta: sense interrupcions imprevistes, podent utilitzar les diverses funcions repetidament, etc.	60%
INTERFÍCIE D'USUARI S'ha tingut cura que tots els components o aspectes de l'aplicació que realitzin algun tipus d'interacció amb l'usuari tinguin una bona usabilitat i un bon nivell de qualitat en la seva presentació, disseny i claredat en la presentació de dades i resultats (valoració del disseny del frontend HTML de l'aplicació)	10%
ESTRUCTURACIÓ DEL CODI El codi del programa està ben organitzat: les operacions que es realitzen es fan en un ordre lògic i correcte, no es repeteix codi d'una manera innecessària, no hi ha estructures de codi incoherents, les operacions es fan d'una forma mínimament òptima i eficient, etc.	10%
FIABILITAT I ROBUSTESA Hi ha una bona comprovació de les situacions d'error que pot experimentar l'aplicació i hi ha una bona resposta en aquests casos (ja sigui intentant recuperar-se de la situació d'error, o bé donant un missatge explicatiu sobre l'error que s'ha produït i les seves causes).	10%
PRESENTACIÓ DEL CODI Claredat, llegibilitat i presentació del codi font del programa: bona tabulació del codi, documentació del codi fent servir comentaris clars, concisos i útils, utilitzar noms de variables que ajudin a entendre la seva utilitat i amb valors inicials explícits quan sigui necessari, comentaris d'encapçalament de mètodes i classes, etc.	10%

SERVIDOR WEB RPI

FASE 3

F3.1

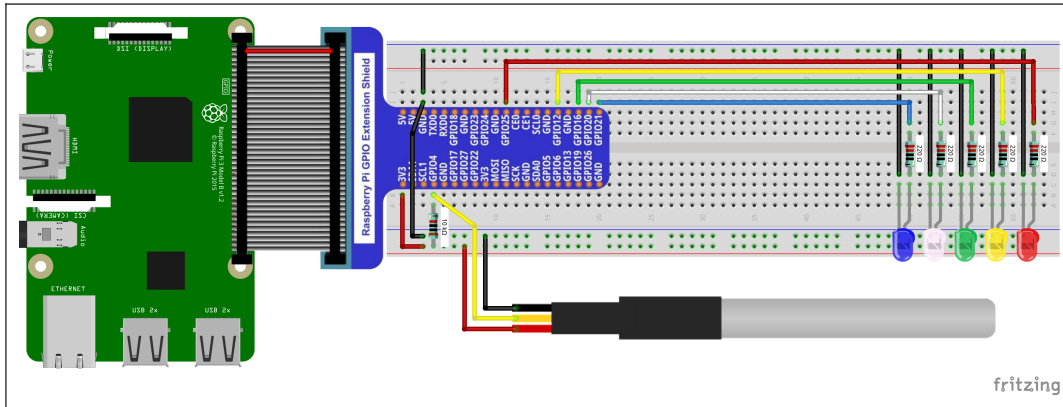
SERVIDOR.V5: més nivells d'indicació de temperatura

Suposem que el nostre servidor monitor de temperatura el volem utilitzar en un centre de dades per fer que ens indiqui si la temperatura de la nau de servidors és la idònia, o no. En primer lloc, volem que hi hagi cinc nivells de temperatura per cadascun dels quals caldrà encendre un indicador lluminós d'un color diferent:

- Si la temperatura està per sota dels 16°C, s'haurà d'encendre un indicador lluminós de color blau.
- Si la temperatura és major o igual que 16°C, però inferior a 19°C, s'haurà d'encendre un indicador de color blanc.
- Si la temperatura és major o igual que 19°C i menor o igual que 22°C, s'haurà d'encendre un indicador de color verd (seria el marge de temperatura ideal).
- Si la temperatura és superior als 22°C i inferior a 27°C, s'haurà d'encendre un indicador de color groc.
- Si la temperatura és superior o igual que 27°C, s'haurà d'encendre un indicador lluminós de color vermell.

Per implementar aquesta millora i fer que el nostre Raspberry Pi pugui fer aquesta tasca de monitoratge de nivells de temperatura, seguiu els passos que s'expliquen a continuació:

- a. En primer lloc, necessitem modificar el muntatge electrònic que teníem fins ara fent que passi a ser com el que es pot veure en el diagrama següent:



- b. Un cop feta la modificació del circuit electrònic, apliqueu els canvis pertinents al codi del procés que va llegint els valors del sensor de temperatura per fer que, en comptes d'encendre només el LED vermell que teníem inicialment, en funció d'un únic llinar de temperatura, ara s'encenguin un dels cinc LEDs que tenim segons els intervals de temperatura que hem indicat.
- c. Modifiquem, també, el codi que dona resposta a la petició de visualització de la temperatura via protocol HTTPS per fer que el valor de la temperatura es mostri a la finestra del navegador en els mateixos cinc colors que els LEDs del circuit segons les mateixes franges de temperatura.



RQ.06 Un cop aconseguiu que funcioni correctament la versió 5 del servidor, amb el monitoratge dels cinc nivells de temperatura, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

F3.2

SERVIDOR.V6: registre de les temperatures observades

Per finalitzar, afegirem una darrera funcionalitat a la nostra aplicació. Aquesta funcionalitat consisteix a fer que cada cert període de temps, per exemple cada 5 minuts, el nostre servidor es connecti a un servidor de base de dades per registrar la temperatura mesurada pel sensor. D'aquesta manera podrem disposar d'un historial de les temperatures observades durant un període de temps més o menys llarg. Per fer-ho, seguiu els passos que s'indiquen a continuació:

- Feu que el programa principal creï un nou procés que serà l'encarregat de registrar les lectures del sensor a la base de dades. Aquest procés, cada 5 minuts, haurà d'establir una connexió sobre la base de dades i haurà d'inserir un nou registre amb el valor de **temperatura** que el sensor estigui mesurant en aquell moment i, també, la **data** i l'**hora** en què aquesta temperatura ha estat observada.
- La base de dades haurà d'estar en una màquina diferent (fora del Raspberry Pi), i haurà d'estar implementada sobre un SGBD PostgreSQL. Per assegurar que la connexió es pot fer correctament, reviseu la configuració del firewall de l'ordinador on teniu el PostgreSQL (especialment si es tracta d'un sistema Windows) i reviseu la configuració de l'arxiu `pg_hba.conf` o `postgres.conf` (depèn de la versió del servidor PostgreSQL que tinguem) per obrir la possibilitat que la base de dades accepti connexions des de màquines remotes.
- Heu d'implementar el mecanisme de persistència de dades mitjançant l'ús del framework de mapatge objecte-relacional (ORM) SQLAlchemy que hem explicat a M6.UF2.
- És recomanable que, per fer això, modifiqueu l'objecte compartit que feu servir per guardar les mostres de temperatura, i hi afegiu els atributs corresponents per poder guardar la data i l'hora. També podeu fer que el procés que cada 5 segons llegeix el sensor i actualitza aquest objecte sigui l'encarregat d'actualitzar la data i l'hora de cada valor de temperatura.



E.03 / RQ.07 Un cop aconseguíu que funcioni correctament la versió 6 del servidor amb el registre de l'historial de temperatures sobre la base de dades, aviseu el professor perquè us faci la revisió del funcionament amb tots els elements que hem proposat fins ara.

AVALUACIÓ DE LA FEINA FETA EN LA TERCERA FASE DEL PROJECTE

La feina d'aquesta tercera fase del projecte l'heu de continuar fent en els grups de 3 alumnes que s'han designat. A més dels punts de revisió i avaluació intermedis d'aquesta segona fase, en finalitzar l'apartat F3.2 caldrà entregar el projecte Python amb la versió 6 del servidor a través de Moodle. L'avaluació d'aquesta darrera entrega de la fase 3 es farà segons la rúbrica que teniu a continuació:

Aspecte	Ponderació
FUNCIONAMENT El circuit i el programa de control funcionen correctament en tots els aspectes que es demanen a l'enunciat, i ho fan de forma robusta: sense interrupcions imprevistes, podent utilitzar les diverses funcions repetidament, etc.	60%
INTERFÍCIE D'USUARI S'ha tingut cura que tots els components o aspectes de l'aplicació que realitzin algun tipus d'interacció amb l'usuari tinguin una bona usabilitat i un bon nivell de qualitat en la seva presentació, disseny i claredat en la presentació de dades i resultats (valoració del disseny del frontend HTML de l'aplicació)	10%
ESTRUCTURACIÓ DEL CODI El codi del programa està ben organitzat: les operacions que es realitzen es fan en un ordre lògic i correcte, no es repeteix codi d'una manera innecessària, no hi ha estructures de codi incoherents, les operacions es fan d'una forma mínimament òptima i eficient, etc.	10%
FIABILITAT I ROBUSTESA Hi ha una bona comprovació de les situacions d'error que pot experimentar l'aplicació i hi ha una bona resposta en aquests casos (ja sigui intentant recuperar-se de la situació d'error, o bé donant un missatge explicatiu sobre l'error que s'ha produït i les seves causes).	10%
PRESENTACIÓ DEL CODI Claredat, llegibilitat i presentació del codi font del programa: bona tabulació del codi, documentació del codi fent servir comentaris clars, concisos i útils, utilitzar noms de variables que ajudin a entendre la seva utilitat i amb valors inicials explícits quan sigui necessari, comentaris d'encapçalament de mètodes i classes, etc.	10%